

Microprocessor based System Design Laboratory (Gy)

Department of Computer Science and Technology, IEST Shibpur

Date: July 23, 2026

Experiment No. 1: Familiarization with the Development Environment

Objective: To be familiar with microprocessor based System Development KIT (SDK).

Goals	Remarks		Time-frame
Identifying the major physical components of the SDK (including the Chips). Eg, The Microprocessor, Memory chips, IO (input output) chips , etc.	Please consult the SDK User Manual (GSAS 85A User's Manual). You have both hard and soft copy of the manual.	A surface level understanding is expected for the time being. Please consult the SDK User Manual (GSAS 85A User's Manual) for this. You have both hard copy and soft copy of the manual.	Today's laboratory class.
To load a machine language program in the main memory of the SDK.		Monitor routine provides the functionality of interactive modification of the memory contents through keypad.	Today's laboratory class.
To execute a program which has already been loaded in the main memory at some known address.		Monitor routine provides the functionality that takes the starting address of the program and then executes it.	Today's laboratory class.
To know the functions (subroutines) which are included in the monitor program and can be called from within user programs,		These subroutines are documented in the SDK manual.	These functions will be required through subsequent classes.
The tasks that we are to try in today's class have been listed at the end of this document.			

General Discussion

In general, **SDK** is a platform for developing **Microprocessor Based Systems**. Like, for development of **software applications**, you have your **laptop** (together with its **Operating System, Compiler** (gcc), **Debugger** (gdb), etc.). In this class you are required to get familiar with the given SDK so that you can use it for development of 8085 based systems.

For microprocessor based system design we need a minimum hardware environment which uses a particular processor (Eg, Intel 8085 in our case) and some supporting components like memory (both **RAM** and **EPROM**), **I/O** ports, buffers etc. This environment is commercially available as a platform for the small system design and development and known as System Development Kit (or simply, **SDK** or **KIT**). Normally, a small program (called **Monitor Program/Software**) is preloaded in the EPROM of the kit. Monitor program is like the operating system (Eg, Microsoft Windows, Linux, etc.) of our Laptop or Desktop computers. When such a kit is powered on, its processor

starts executing the monitor program, which in turn interacts with the user through the keypad/display for loading, running and debugging user programs.

In this laboratory class you will be working with 8085 Microprocessor based SDK (Model GSAS 85A from GSAS Micro Systems Pvt. Ltd.). You will be provided with one such SDK along with its User Manual.

For development of programs using this **SDK**, you may -

1. first write (pen and paper) **8085 Assembly Language** programs using the instructions from **8085 Instruction Set** (soft copy provided),
2. and then consult the Instruction Set to manually translate (pen and paper) the **Assembly Language Program** into **8085 Machine Language**.
3. This machine language program then can be **“loaded” into the (RAM section of the) main memory of the SDK with the help of the monitor program through the keypad.**
4. Finally the user may execute the program (s)he has loaded at the given location of the main memory again **with the help of the monitor program through the keypad..**

Always follow the following format for writing your programs.

For example, the following table contains a 8085 program (in **assembly** as well as **machine language**) with documentation (comments).

Label (Name)	Assembly Instruction	Chosen Memory Address (Hex)	Machine Language (Hex)	Comment
	MVI B, 22H	8800	06	Register B is initialized with 22H
		8801	22	
	MVI C, 82H	8802	0E	Register C is initialized with 82H
		8803	82	
	MOV A, B	8804	78	Contents of B (that is, 22H) is copied to A
	STA 8840H	8805	32	Contents of A is stored (copied) to Memory location 8840H
		8806	40	8085 is Little Endian , and hence, in the main memory the lower order byte of 8840H, that is, 40H comes first before 88H, the higher order byte.
		8807	88	
	HLT	8808	76	

After execution (ie, **GO-8-8-0-0-EXEC**) of the above program you may check the contents of the memory location @8840H (should contain 22H). Please note that the last instruction of the program is HLT (halt) and, hence, the 8085 processor of your SDK will halt after execution of the program, and the SDK will become non-interactive. You have to press the **RESET** button to make it execute the Monitor program and make the SDK interactive again.

SDKs also provide a set of “connectors” using which one can connect external Input/Output devices.

Specification of a Typical KIT (Your KIT may have different specification and please check what your kit actually has.):

- The Microprocessor - A CPU (8/16 bit popular general purpose CPU).
- Memory: RAM – 2K, EPROM – 2K. Memory may be enhanced 4K/8K or more.
- Man-Machine Interface: Input: Keypad with 24 (or more) keys; Output: 7 segment LED Display Panel with 6 or more display modules.
- I/O ports: 2 to 8 through standard Programmable Input Output (PIO) devices for I/O operations and peripheral interfaces.

- Communication : RS-232C compatible (e.g., a PC may be connected to the KIT through this serial channel) serial line.
- Monitor routine: Machine language routine (1K–2K in size) Residing in EPROM.
- Optional features: On-board EPROM programmer, Analog to Digital Conversion & Digital to Analog Converter etc.
- Storage (Secondary) : None

Facilities in the KIT: The keyboard along with the LED display unit serve as basic I/O media for communicating with the user. Programs are loaded in the RAM (user area) through key-press; and executed. Debugging, if required, is also done by using appropriate command/numeric key(s). The following can be done in a KIT:

- i. Reset the system (Cold Start).
- ii. Reset the system with preservation of processor status in memory (Warm Start).
- iii. Modify/examine the content of a memory location.
- iv. Execute a program.
- v. Debug a program by verifying/changing the contents of the accessible registers.
- vi. Set breakpoints and run the program in single step mode.

Tasks you should attempt

Get familiar with the SDK and its manual given to you._

1. Identify different hardware components
 1. **CPU** (the 8085 chip)
 2. **Main Memory** (ROM and RAM sections) – the chips, memory size, where in the memory space (0000H to FFFFH) they are interfaced. (Ref. Chapter 5, page 5-2 of the SDK user manual)
 3. **Input/Output (IO) devices** – The devices, chips, and where where in the memory space (0000H to FFFFH) or IO space (00H to FFH), they are interfaced. (Ref. page 5-3 to 5-4 of the SDK user manual)
2. How to use (interact with) the monitor program of the SDK. (Ref. Chapter 3, page 3-3 of the SDK user manual)
 1. Examine/Modify Memory – mainly to be used to load/modify programs
 2. Examine/Modify Register
 3. Single Instruction or Single Stepping – Used to debug a program
 4. Execute a program which has already been loaded in the memory
 5. Block Move
 6. Delete
 7. Insert
 8. etc.
3. **Load and execute the sample programs given in Chapter 14 of the Manual.**

Report: The format of the Laboratory Report and the submission mechanism will be stated during class hours.